

PO4116r0 | Completing support for emotive programming in C++

Pablo Halpern phalpern@halpernwrightsoftware.com

2016-04-01 | Target audience: Evolution Working Group (EWG)

1 Motivation

There can be no doubt that C++ is an unusually expressive language. A significant part of that expressiveness comes from the ability to use emoticons, not just in comments and string literals, but in the code itself. For example the following (do-nothing) program compiles, links, and runs in all versions of C++:

```
int main(int argc, char* argv[])
{
    int D = 0, P = 0, O = 0, J = 0, I = 0, _8 = 0, heart = 0;
#    define C +D

// Name          emoticon          suffix
// -----
    laughing      :-D              ;
    sad           :- (             0);
    tongue        :-P              ;
    wink:         ;-D              ;
    glasses:       8-D             ;
    surprise       :-0             ;
    kiss           :-*             &D;
    tongue_in_cheek :-J             ;
    heart          <3              ;
    homer_simpson: ~(_8^(I)        );

    I_love_cpp:    I <3 C++        ;
}
```

Notably absent from the emoticons available to the C++ programmer is the most common emoticon of all: “:-)”. There is simply no valid code where the three tokens can appear in sequence. This omission has been generating complaints for years, and is even more glaring as emotive programming becomes more established and takes its place alongside object-oriented programming, generic programming, and stream-of-consciousness programming, all of which are paradigms currently supported in C++. We (and by “we” I mean, “I”) believe it is time to correct this flaw and make the canonical emoticon a legal token sequence in the language.

2 Proposed solutions

We propose making :-) a single token in the language. Unfortunately, that means that :) would still not be accepted. We consider this to be an acceptable compromise because studies show that people who use :) are less intelligent than those who use :-), and we can assume that they are not programming in advanced C++.

The community is split on the best meaning for this new token. Two of the most popular (but by no means only) meanings are described in **Option #1** and **Option #2**, below.

2.1 Option #1: Literal value of a unique type

Like `nullptr`, `:-)` could be a reserved word representing a literal value of unique type. Such a type could then be used for overloading, as follows:

```
namespace std {
    typedef decltype( :- ) emoticon_t;
}

void foo(int);
void foo(std::emoticon_t);

foo( :- ) ; // Call foo(std::emoticon_t)
```

The standard library would overload `ostream::operator<<(emoticon_t)` to print out the emoticon, itself, or an equivalent emoji, as determined by an obscure setting in `locale`.

2.2 Option #2: Overloadable operator

Our preferred option is to make `:-)` an overloadable operator with no default meaning, not even for built-in types. The precedence of the operator would logically be between the precedence of `operator-` and `operator()`. Both unary-postfix and binary overloads would be permitted. A common use would be as a “just kidding” operator to, for example, undo a previous operation that wasn’t really intended. The impact of its use would be to make code less offensive. Applying unary `operator:-)` to an `chrono::time_point` value would return the date, April 1, 2016. An example of its use would be:

```
void operator :- ) (void (*)()); // Unary postfix operator

if (programming_experience == "Java")
    terminate :- ) ;
```

3 Alternatives considered

3.1 Rejected alternative #1: Give a meaning to - followed by)

A minimal approach would be to allow an unbalanced closing parenthesis to have a meaning when preceded by a dash. While workable linguistically, it is far from clear what meaning can be ascribed to such a construct. Moreover, unbalanced parenthesis cause problems for text editors, which already have difficulties matching up angle brackets in templates. In contrast, a text editor can easily recognize `:-)` as a single token, and not try to match it to an open parenthesis. (We use the term, “easily” here to mean anything solvable in polynomial time or better.)

3.2 Rejected alternative #2: Use unicode emoji to show emotion in code

It was pointed out by our smart alec summer intern that Unicode has many emoji, useful for expressing almost every kind of emotion. If these emoji could be used as either identifiers or overloadable operators, there would be no need for a `:-)` emoticon. We find this to be an unacceptable solution for several reasons:

1. The existing emoticons in common use by C++ programmers do not use characters outside the basic character set.
2. Once Unicode is allowed, people will begin to question why `@` and `$` are not permitted – a question we *do not* want asked (because we don’t know the answer).

3. The idea of Unicode in C++ is, frankly, preposterous.

4 Impact on the standard

We do not yet have formal wording, but the expected impact includes:

- Numerous examples improved through use of : -)
- A happier and more amusing standard, in general
- Greater acceptance of C++ for facetious applications

5 Implementation experience

Our experience is that implementing operator : -) is a lot of fun.

6 References

An Introduction to Emotive Programming, Jo King and Kih Ding Yu, OOPSLA, April 1, 2014

Emoticons in the Age of Aquarius, Justin Funn, Journal of Spiritual Computing, April 1, 1983